Guía de repaso — Examen Redes I (Wireshark + libpcap/Python)

---

1. Wireshark: la ventana principal

Se divide en 3 partes:

Parte	Qué muestra
Arriba — lista de paquetes	No., Time, Source, Destination, Protocol, Length, Info
Medio — árbol de capas	Detalle del paquete seleccionado, expandible por capa (Ethernet → IP → TCP/UDP → …)
Abajo — hexadecimal	Bytes en crudo; al hacer clic en un campo del medio, se resaltan sus bytes aquí

Cómo abrir Wireshark en la VM: - Traza guardada: menú de aplicaciones → Internet → Wireshark - Captura en vivo: `sudo wireshark` (necesita permisos root para acceder a la interfaz)

---

2. Filtros: captura vs visualización (¡no los confundas!)

	Filtro de captura	Filtro de visualización
¿Cuándo actúa?	ANTES de capturar (descarta a nivel de sistema)	DESPUÉS de capturar (solo oculta, no borra)
¿Dónde se pone?	Ventana de opciones de captura (icono engranaje)	Barra superior "Aplique un filtro de visualización"
Sintaxis	Estilo BPF: `tcp`, `udp`, `host 1.2.3.4`, `port 80`	Sintaxis Wireshark: `tcp.port == 80`, `ip.addr == 1.2.3.4`
Efecto en el fichero guardado	Los paquetes descartados NUNCA se guardan	Todos los paquetes siguen en el fichero, solo se ocultan de la vista

Filtros de visualización más usados

```
tcp                                 # solo TCP
udp                                 # solo UDP
dns                                 # solo DNS
arp                                 # solo ARP
icmp                                # solo ICMP/ICMPv6

ip.addr == 192.168.1.1              # tráfico hacia/desde esa IP
ip.src == 192.168.1.1               # solo como origen
ip.dst == 192.168.1.1               # solo como destino

tcp.port == 80                      # puerto 80, origen o destino
tcp.srcport == 80                   # puerto 80 solo como origen
```

```
tcp.dstport == 80                    # puerto 80 solo como destino
udp.srcport == 53                    # respuestas DNS (puerto origen
53)

tcp.flags.syn == 1 && tcp.flags.ack == 0   # solo SYN puros
tcp.flags.syn == 1 && tcp.flags.ack == 1   # solo SYN-ACK
tcp.flags.reset == 1                       # solo RST

frame.len > 1000                     # paquetes grandes
frame.len == 95                      # tamaño exacto

ip && frame.len > 1000               # combinar con &&
!udp                                 # negar con !
```

Filtro de captura (solo tráfico UDP, por ejemplo)

En la ventana de “Opciones de captura”, campo “Capture Filter”:

```
udp
```

Esto hace que ni siquiera se capturen paquetes TCP/ARP — quedan totalmente fuera, no solo ocultos.

2bis. Filtros de captura + comando para generar ese tráfico

IMPORTANTE: los filtros de captura van en MINÚSCULAS y en sintaxis BPF (texto plano tipo `and`/`not`/`host`/`port`), NUNCA `&&` ni mayúsculas — si escribes `ICMP` o `TCP.PORT==80` te dará “syntax error”.

Filtro de captura	Qué captura	Comando en terminal para generarlo
`icmp`	Pings (ICMP echo request/reply)	`ping -c 5 www.uam.es`
`arp`	Peticiones/respuestas ARP	`sudo ip neigh del <IP_gateway> dev ens33` seguido de `ping -c 2 <IP_gateway>`
`udp`	Todo el tráfico UDP (DNS, NTP, QUIC…)	`ping -c 3 www.uam.es` (genera DNS) o abrir Firefox
`tcp`	Todo el tráfico TCP	Abrir Firefox y navegar, o `sudo hping3 -S -p 80 www.uam.es`
`port 53`	Tráfico al/del puerto 53 (DNS)	`nslookup www.uam.es` o `ping www.uam.es` (resuelve el nombre primero)
`port 80`	Tráfico al/del puerto 80 (HTTP)	`sudo hping3 -S -p 80 www.uam.es`
`host <IP>`	Tráfico hacia/desde una IP concreta	`ping -c 3 <IP>`
`tcp port 80`	Solo TCP en el puerto 80 (combinado)	`sudo hping3 -S -p 80 www.uam.es`
`not arp`	Todo menos ARP	Abrir Firefox y navegar (genera DNS/TCP, sin forzar ARP)
		Cualquier tráfico

`net 192.168.1.0/24`	Todo el tráfico de esa subred	normal dentro de tu red local

---

Truco para el examen: si el profesor te pide “captura solo tráfico X”, primero identifica el filtro de captura correcto (tabla de arriba), y LUEGO piensa qué comando de terminal generaría ese tipo de tráfico para comprobar que funciona — son dos pasos distintos, no los mezcles.

¿Qué es `<IP_gateway>` y cómo la saco?

Cualquier cosa escrita entre < > en esta guía (o en cualquier documentación técnica) es un **marcador de plantilla**: significa “sustituye esto por tu valor real”, nunca escribas los símbolos < y > de forma literal.

`<IP_gateway>` es la **puerta de enlace** — la IP del router al que tu máquina manda el tráfico cuando quiere salir de tu red local.

Pasos para averiguarla: 1. Abre una terminal en la VM. 2. Ejecuta: `ip route` 3. Busca la línea que empieza por `default via` — el número justo después de `via` es tu gateway. Por ejemplo: `default via 192.168.154.2 dev ens33 ...` Aquí, la IP de gateway es `192.168.154.2`. 4. Sustituye ese número donde ponía `<IP_gateway>` en los comandos. Por ejemplo: `sudo ip neigh del 192.168.154.2 dev ens33  ping -c 2 192.168.154.2`

Ten en cuenta que **tu IP de gateway puede cambiar** cada vez que reinicies la VM o te reconectes a la red — por eso conviene comprobarla con `ip route` cada sesión, en vez de memorizar un número fijo.

---

3. Las 3 pestañas de “banderas” (flags) TCP más importantes

Flag	Significado
SYN	“Quiero abrir una conexión” (primer paso del handshake)
ACK	“Confirmo que he recibido algo”
SYN, ACK	Respuesta del servidor aceptando la conexión
RST	“Corto la conexión de golpe” (ej: puerto cerrado, o `hping3` rechazando un SYN-ACK inesperado)
FIN	“Quiero cerrar la conexión de forma ordenada”

Patrón típico de un escaneo o de hping3: SYN → (SYN,ACK si el puerto está abierto) → RST inmediato (porque el kernel no reconoce esa conexión como iniciada por él).

---

4. Cabeceras — la parte que más cae en examen

Ethernet (14 bytes fijos)

Campo	Bytes	Posición
MAC Destino		

	6	0-5
MAC Origen	6	6-11
EtherType	2	**12-13**

EtherTypes que hay que saberse de memoria: | Valor hex | Protocolo | |—|—| | `0x0800` | IPv4 | | `0x0806` | **ARP** | | `0x86DD` | IPv6 |

(Byte 12 = `08`, byte 13 = `06` → juntos forman `0x0806` = ARP. Esto es justo lo que usamos en la práctica.)

IP (mínimo 20 bytes, sin opciones)

Campos clave: Versión, IHL (longitud de cabecera), Longitud Total, TTL, Protocolo (6=TCP, 17=UDP, 1=ICMP), Dirección Origen, Dirección Destino.

Relación importante: `Length` (columna Wireshark, trama completa) = `Total Length` del IP + 14 (la cabecera Ethernet que va por fuera del IP).

TCP (mínimo 20 bytes)

Campos clave: Puerto Origen, Puerto Destino, Número de Secuencia, Número de ACK, Flags (URG,ACK,PSH,RST,SYN,FIN), Ventana.

---

5. Modo promiscuo

- **Activado (1):** la tarjeta de red procesa TODO el tráfico que le llega, vaya dirigido a ella o no.
- **Desactivado (0):** solo procesa lo dirigido a su propia MAC.
- Necesario para poder "ver" tráfico ajeno con Wireshark/pcap. En switches modernos influye poco (el switch ya filtra), pero sigue siendo el modo estándar a usar.

---

6. Interfaces típicas en Linux

Nombre	Qué es
`lo`	Loopback — tráfico interno de la propia máquina (127.0.0.1)
`ensX` / `ethX`	Tarjeta de red Ethernet (cableada)
`wlanX`	Tarjeta WiFi
`any`	Captura de todas las interfaces a la vez

---

7. Repaso del código — funciones clave de `rc1_pcap.py`

Función	Para qué sirve
`pcap_open_live(dev, snaplen, promisc, to_ms, errbuf)`	Abrir una interfaz en vivo (necesita sudo)
`pcap_open_offline(fname, errbuf)`	Abrir una traza `.pcap` ya guardada
`pcap_loop(handle, cnt, callback, user)`	Bucle que llama a `callback` por cada paquete (`cnt=-1` = ilimitado)
	Crear una "plantilla" para un

`pcap_open_dead(linktype, snaplen)`	fichero de salida (linktype=`DLT_EN10MB`=1 para Ethernet)
`pcap_dump_open(descr, fname)`	Abrir de verdad el fichero de salida, devuelve un “dumper”
`pcap_dump(dumper, header, data)`	Guardar UN paquete en el fichero
`pcap_close(handle)`	Cerrar el descriptor de lectura
`pcap_dump_close(dumper)`	Cerrar el fichero de escritura
`pcap_breakloop(handle)`	Parar `pcap_loop` de forma ordenada (usado en el `signal_handler` de Ctrl+C)

Estructura del `header` que llega en el callback: - `header.len` → tamaño real del paquete - `header.caplen` → tamaño realmente capturado (puede ser menor que `len` si se truncó) - `header.ts.tv_sec` / `header.ts.tv_usec` → timestamp en segundos + microsegundos

---

8. Repaso del código — conceptos Python que usamos

Concepto	Para qué
`argparse`	Leer argumentos de la línea de comandos (`--itf`, `--file`, `--nbytes`, `--npkts`)
`dest='x'` en `add_argument`	Nombre de la variable donde se guarda el valor (`args.x`)
`default=...`	Valor si el usuario no pasa esa opción
`type=int`	Convierte el texto recibido a número entero
`action='store_true'`	Para flags booleanos tipo `--debug` (sin valor detrás)
`global` dentro de una función	Necesario si vas a MODIFICAR una variable definida fuera de la función
`logging.debug/info/error(...)`	Imprime mensajes con nivel de importancia; se filtran según `logging.basicConfig(level=...)`
`binascii.hexlify(bytes)`	Convierte bytes a su representación hexadecimal en texto
`data[i]`	Accede al byte en la posición `i` de un paquete (empezando en 0)
`sys.exit(-1)`	Termina el programa con código de error
`signal.signal(signal.SIGINT, handler)`	Conecta Ctrl+C con una función propia

El “único flujo” que pedía el enunciado:

```
        if args.interface is not False:
            handle = pcap_open_live(...)
            # + crear los 2 dumpers
        else:
            handle = pcap_open_offline(...)
        # A PARTIR DE AQUÍ, código único: pcap_loop, procesa_paquete,
cierre...
```

---

9. Posibles “trampas” que te puede poner el profesor en el examen

- Pedirte cambiar el criterio de separación de tráfico (otro EtherType, u otra condición sobre otros bytes).
- Pedirte añadir un nuevo argumento (`--algo`) siguiendo el mismo patrón de `add_argument`.
- Preguntarte qué pasaría si quitas el `global` de una variable (dejaría de modificar la de fuera, crearía una copia local).
- Preguntarte por qué se usa `min(n, len(data))` (evitar `IndexError` si pides más bytes de los que el paquete tiene).
- Pedirte un filtro concreto de Wireshark de memoria (repasa la sección 2).
- Preguntar la diferencia entre `pcap_close` y `pcap_dump_close` (uno cierra lectura, otro cierra escritura).
- Preguntar qué bytes corresponden al EtherType y qué significan `0x0800/0x0806/0x86DD`.

---

10. Comandos de terminal que usaste y conviene recordar

```
        sudo python3 practica1.py --itf ens33 --npkts 20 --nbytes 40
        sudo wireshark
        ip addr                          # ver tus interfaces e IPs
        ip route                         # ver tu puerta de enlace
        sudo ip neigh del <IP> dev ens33 # borrar una entrada ARP en caché
        ping -c 3 <IP>                   # forzar tráfico (y ARP si hace
falta)
```